

Here's a visual breakdown of **JavaScript hoisting**, showing what happens during the creation phase vs. execution phase.

What is Hoisting?

Hoisting is the behavior where JavaScript creates bindings for declarations during the **scope setup / creation phase**, before executing the code line-by-line.

Important: Don't imagine JavaScript literally moving your declarations to the top. Think: **bindings are created before execution**.

1 Creation / Setup Phase

`var x` → created + initialized to **undefined**

`let y` → created, but **uninitialized**

`const z` → created, but **uninitialized**

2 Execution Phase

`x = 10` → assignment happens

`y = 20` → initialization happens

`z = 30` → initialization happens

Example 1 — `var`

```
console.log(x); // undefined
var x = 10;
console.log(x); // 10
```



Setup: `x` → **undefined**

Execute: `console.log(x)` → **undefined**

Then: `x = 10`

Example 2 — `let`

```
console.log(y); // ❌ ReferenceError
let y = 20;
```



Setup: `y` exists but is **uninitialized**

Before `let y = 20`: TDZ

Access:  `ReferenceError`

Example 3 — Function Declaration

```
greet(); //  "Hello"

function greet() {
  console.log("Hello");
}
```



Function declarations are available before their position in the source code during execution.

 Function Expression is different

```
greet(); //  TypeError

var greet = function () {
  console.log("Hello");
};
```

`var greet` is initialized to `undefined`. Therefore `greet()` tries to call `undefined`.

Mental model

Creation → bindings created → Execution →
declarations initialized / values assigned

`var` → created + **undefined**

`let` / `const` → created + **TDZ** until initialization

function declaration → function available

JavaScript Hoisting

Hoisting means:
"Bindings are created before code is executed,
not that the code is physically moved."



How it works?

- 1 During the **creation phase**, JavaScript scans the code and creates bindings for variables and functions.
- 2 Then it executes the code line by line.
- 3 That's why you can sometimes use variables/functions before their declaration (depending on var, let, const).

Two Phases of Execution Context

1. Creation Phase (Memory Setup)

- Create bindings for variables and functions.
- var → initialized with undefined
- let / const → created but uninitialized (TDZ)
- Function declarations → fully initialized with function body

2. Execution Phase (Run code line by line)

- Execute statements
- Assign values
- Initialize let / const
- Run functions

1. var Hoisting

```
1 console.log(x); // undefined
2 var x = 10;    // ...
3 console.log(x); // 10
```

Creation Phase (before execution)

Global Execution Context

x : undefined
← Binding created & initialized with undefined

Execution Phase (line by line)

```
1 console.log(x); // undefined
2 x = 10;         // assign value
3 console.log(x); // 10
```

Output

undefined
10

2. let Hoisting (TDZ)

```
1 console.log(y); // ReferenceError
2 let y = 20;
3 console.log(y);
```

Creation Phase (before execution)

Global Execution Context

y : <uninitialized>
Binding created but not initialized (Temporal Dead Zone)

Execution Phase (line by line)

```
1 console.log(y); // ❌ ReferenceError
2 y = 20;         // initialize
3 console.log(y); // 20
```

Output

ReferenceError: Cannot access 'y' before initialization

3. const Hoisting (TDZ)

```
1 console.log(z); // ReferenceError
2 const z = 30;
3 console.log(z);
```

Creation Phase (before execution)

Global Execution Context

z : <uninitialized>
Binding created but not initialized (Temporal Dead Zone)

Execution Phase (line by line)

```
1 console.log(z); // ❌ ReferenceError
2 z = 30;         // initialize
3 console.log(z); // 30
```

Output

ReferenceError: Cannot access 'z' before initialization

4. Function Hoisting

```
1 greet(); // Hello!
2 function greet() {
3   console.log("Hello!");
4 }
```

Creation Phase (before execution)

Global Execution Context

greet : [Function]
Entire function is available

Execution Phase (line by line)

```
1 greet(); // Hello!
           (function already available)
2 function greet() { ... } // no effect
           (already created)
```

Output

Hello!

Function Expression (Not Hoisted like function declaration)

```
1 sayHi(); // TypeError
2 var sayHi = function() {
3   console.log("Hi");
4 };
```

Creation Phase

sayHi : undefined

Execution Phase

```
1 sayHi(); // ❌ TypeError
2 sayHi = function() { ... }
```

Output

TypeError:
sayHi is not a function

Key Takeaways

- ✓ Hoisting is about bindings, not moving code.
- ✓ var → hoisted and initialized with undefined.
- ✓ let / const → hoisted but in TDZ (cannot access).
- ✓ Function declarations → fully hoisted.
- ✓ Function expressions → behave like variables (e.g., undefined initially with var).
- ✓ Always declare variables at the top (good practice).

"Same concept, different behavior. Understand the phase, not just the output!"

